Software Engineering Guess Paper

1. **What is the prime objective of software engineering?**

The prime objective of software engineering is to develop high-quality software that meets the needs of its users, is reliable, efficient, and maintainable. ● Problem-solving and decision-making ● Analytical skills:

- Collaboration:
- Continuous learning:

  Project management:

2. **What do you mean by formal requirement specification?**

Formal requirement specification is a detailed, clear, and concise description of the requirements for a system or project. It typically involves a written description of the invention, enablement, and best mode of carrying out the claimed invention, expressed in a mathematical notation with precisely defined vocabulary, syntax, and semantics.

3. **Define design pattern and modularity in object oriented design?**

Design pattern is a reusable approach to common programming challenges, and modularity in object-oriented design refers to an organizing structure in which different components of a software system are divided into separate functional units.

4. **Mention the key elements of software engineering.**

Key elements of software engineering include requirements analysis, design, implementation, testing, and maintenance.

5. **What is the objective of various model in software engineering?**

The objective of various models in software engineering is to provide a structured approach to software development, allowing developers to plan, design, and implement software systems efficiently and effectively.

6. **Why accuracy is important in the data dictionary?**

Accuracy is important in the data dictionary to ensure that the data being used is reliable and consistent, reducing the risk of errors and improving the overall quality of the software.

7. Difference between LOC and FP estimation.

LOC (Lines of Code) estimation is a measure of the size of a software system, while FP (Function Point) estimation is a measure of the functionality provided by the software.

8. **Why design document is important in software engineering?**

The design document is important in software engineering as it provides a detailed description of the software system, including its architecture, design, and functionality, helping developers to understand the system and its requirements.

9. **What is component diagram in the context of UML?**

A component diagram in UML (Unified Modeling Language) is a graphical representation of the components of a software system and their relationships, helping developers to understand the system's structure and organization.

10. **What is finite state machine.**

A finite state machine is a mathematical model used to describe the behavior of a system that can be in one of a finite number of states at any given time.

11. **Define project scheduling in software engineering.**

Project scheduling is the process of planning and organizing the tasks and resources required to complete a project, ensuring that it is completed on time and within budget.

12. **What is data modelling in software engineering?**

Data modeling in software engineering is the process of creating a conceptual model of the data required by a software system, helping developers to understand the data and its relationships.

13. **List the stages of object-oriented design.**

The stages of object-oriented design include requirements analysis, design, implementation, testing, and maintenance.

14. **Define the design principles**

Design principles are general guidelines and best practices used to create software that is maintainable, scalable, and efficient. They help ensure that software is easy to understand, modify, and maintain, reducing the likelihood of bugs and improving overall quality.

15. **Identify requirement analysis task**

Requirement analysis is the process of identifying and documenting the requirements for a software system, ensuring that it meets the needs of its users.

16. **Differentiate Testing and Debugging**.

Testing and debugging are two distinct processes in software development. Testing involves verifying that the software meets its requirements, while debugging involves identifying and fixing errors in the code.

17. **Explain the feasibility phase of software life Cycle Model.**

The feasibility phase of the software life cycle model is used to determine whether a

proposed software system is technically and economically viable, considering factors such as cost, time, and resources.

18. **Discuss process specification?**

Process specification is a detailed description of the processes and procedures used in software development, ensuring that the software is developed efficiently and effectively.

19. **Discuss the characteristics of Good Design?**

A software product can be judged by what it offers and how well it can be used. This software must satisfy on the following grounds:

Ø Operational

· Budget · Usability · Efficiency · Correctness · Functionality · Dependability · Security · Safety Ø
Transitional
· Portability · Interoperability · Reusability · Adaptability

Ø Maintenance
· Modularity · Maintainability · Flexibility · Scalability

20. **Describe effort estimation?**
Effort estimation is the process of estimating the resources and time required to complete a software project, helping developers to plan and manage the project effectively.

21. **Explain behavior modelling in software engineering.**
Behavior modeling in software engineering is the process of creating a model of the behavior of a software system, helping developers to understand how the system will respond to different inputs and situations.

22. **What are differences between verification and validation?**
Verification is the process of checking that the software meets its requirements, while validation is the process of checking that the requirements themselves are correct and complete.

23. **Define system engineering?**
System engineering is the process of designing and developing complex systems, including both hardware and software components.

24. **Define the merits of the various model in software engineering.**
The merits of various models in software engineering include their ability to provide a structured approach to software development, improve collaboration and communication among team members, and reduce the risk of errors and rework.

25. **Difference between RAD and incremental model.**
RAD (Rapid Application Development) and incremental models are two different approaches to software development, with RAD focusing on rapid prototyping and iteration, while the incremental model involves breaking the project into smaller, more manageable tasks.

26. **Discuss about the functional and non-functional requirements.**
Functional requirements describe the specific actions that the software must perform, while non-functional requirements describe the qualities or characteristics of the software, such as performance, security, and usability.

27. **Explain Project planning of Software Management Activities.**
Project planning in software management activities involves creating a plan for the development, testing, and deployment of a software system, including identifying tasks, resources, and timelines.

28. **Explain the term System engineering and Software Engineering.**

System engineering and software engineering are related but distinct fields, with system engineering focusing on the design and development of complex systems, including both hardware and software components, while software engineering focuses specifically on the development of software systems.

29. **Explain phases of prototype and waterfall model?**

The phases of prototype and waterfall model include requirements analysis, design, implementation, testing, and maintenance, with the prototype model allowing for more flexibility and iteration, while the waterfall model follows a more linear, sequential approach.

PART - B

(Word limit 100) (5x4=20)

1. **Describe software requirement specification in detail?**

A Software Requirement Specification (SRS) is a detailed document that outlines the functional and non-functional requirements of a software system. It includes a comprehensive description of the system's features, capabilities, and constraints. SRS serves as a blueprint for the development team, providing a clear understanding of what needs to be implemented. It covers aspects such as system architecture, user interfaces, data structures, and external interfaces. The SRS document plays a crucial role in the software development life cycle by serving as a reference for both developers and stakeholders.

2. **Define spiral model in brief.**

The Spiral Model is a software development model that combines elements of the waterfall model and iterative development. It divides the development process into a series of repeating cycles called "spirals." Each spiral represents a phase in the development process, including planning, risk analysis, engineering, and evaluation. The model allows for incremental releases of the product, with each iteration building upon the previous ones. The Spiral Model is particularly suitable for large, complex projects where risks and uncertainties need to be addressed throughout the development process.

3. **Discuss data and control flow Diagram in detail.**

Data Flow Diagrams (DFD) illustrate the flow of data within a system, while Control Flow Diagrams show the control relationships between system components. In a DFD, processes, data stores, and data flows are represented graphically. Control Flow Diagrams depict the sequence of operations and control structures in a system. Both diagrams aid in understanding the system's architecture, data movement, and control logic.

4. **Explain Design fundamental in brief.**

Design fundamentals in software engineering include principles like modularity, abstraction, encapsulation, and hierarchy. Modularity promotes dividing the system into manageable and independent modules, while abstraction involves hiding unnecessary details.

Encapsulation encapsulates the implementation details within a module, and hierarchy organizes modules in a structured manner.

5. **Explain class and object relationships by using an example.**

Classes and objects are fundamental concepts in object-oriented programming. A class is a blueprint for creating objects, and relationships between classes can include associations, aggregations, and compositions. For example, consider a "Car" class that has an association with a "Driver" class. The relationship could represent that a driver is associated with a specific car.

6. **Elaborate the problem that occurs while developing the system and suggest possible solutions**.

Issues during system development may include scope changes, unclear requirements, and communication gaps. Solutions involve thorough requirement analysis, effective communication channels, and employing agile methodologies to adapt to changes efficiently.

7. **Describe the Computer-Based system as an organizational information system with an example**.

A Computer-Based System (CBS) is an information system that uses computer technology to support organizational processes. An example is an Enterprise Resource Planning (ERP) system, which integrates various business functions like finance, human resources, and supply chain management into a unified platform for efficient data management.

8. **Explain the software development life cycle with a diagram.**

Software Development Life Cycle (SDLC) is a process used for planning, creating, testing, and deploying software systems. The cycle typically includes phases such as planning, analysis, design, implementation, testing, deployment, and maintenance.

9. **What do you understand by data dictionary where and how it is used?**

A Data Dictionary is a centralized repository that stores information about data elements, their characteristics, relationships, and meanings. It is used in database management and software development to maintain consistency and provide a reference for data-related details.

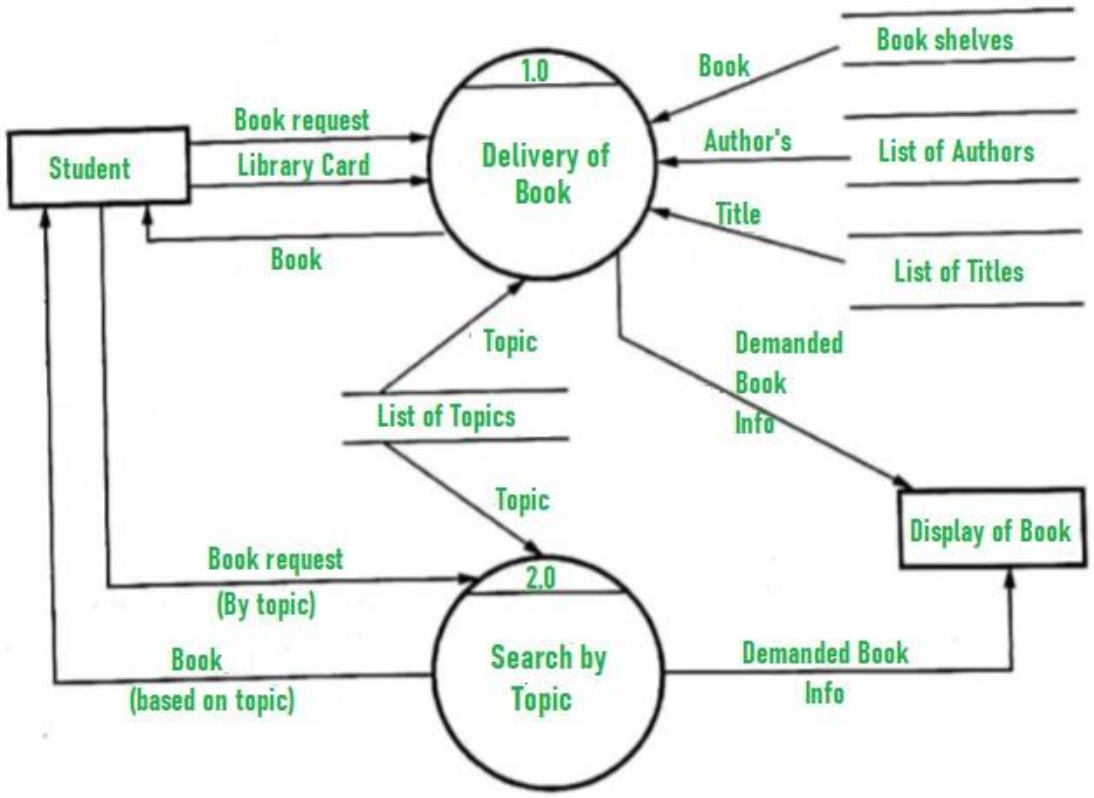
10. **Explain the object modular radiation with an example.**

Object Modularization involves breaking down a system into small, manageable modules, each representing an object. For example, in a payroll system, modules could include "Employee," "Salary," and "Tax." Each module encapsulates its functionality.

11. **Draw DFD in Library management?**

Level 0 DFD:This level provides an overview of the system, showing the main processes and data stores.

- Level 1 DFD:This level provides more detail on the processes and data flows identified in the Level 0 DFD.
- Level 2 DFD:This level further elaborates on the processes and data flows, providing a more detailed view of the system's functionality.



Level 1 DFD

13. **What is UML? Explain how it is useful in object-oriented modeling.**

UML is a standardized modeling language for visualizing, designing, and documenting software systems. It includes various diagrams like class diagrams, use case diagrams, and sequence diagrams, aiding in object-oriented modeling.

14. **Explain software prototyping with its types and example.**

Software Prototyping involves creating a preliminary version of the system for user evaluation. Types include throwaway prototyping and evolutionary prototyping. For example, developing a quick interface mock-up for user feedback.

15. **What is design documentation in software engg. Explain along with its importance in detail.**

Design documentation encompasses documents like system architecture, data models, and interface specifications. Its importance lies in providing a reference for developers, ensuring system consistency, and facilitating maintenance and future enhancements.

1. **Evaluation of Software Life Cycle Model in a Live Project:**

The stages of a software life cycle model, such as Waterfall, Agile, or Spiral, play a crucial role in the successful development of a software project. In a live project, the software life cycle typically involves stages like requirement analysis, design, implementation, testing, deployment, and maintenance. The evaluation of these stages in a live project is essential for ensuring the project's success and meeting user expectations.

- Requirement Analysis: This stage involves gathering and understanding the client's needs. The effectiveness of this stage lies in clearly defining and documenting the project requirements to serve as a foundation for subsequent phases.

- Design: The design phase focuses on creating a blueprint for the software based on the gathered requirements. Evaluation includes assessing the design's clarity, scalability, and adherence to best practices.

- Implementation: This phase involves actual coding based on the design specifications. Evaluation criteria include code quality, adherence to coding standards, and the efficiency of the implementation process.

- Testing: Thorough testing is crucial to identify and rectify defects. The evaluation of this stage includes the effectiveness of test cases, coverage, and the ability to detect and fix bugs.

- Deployment: Deploying the software into a live environment requires careful planning and execution. Evaluation involves assessing the smoothness of the deployment process and monitoring for any unexpected issues.

- Maintenance: Post-deployment, the software enters the maintenance phase to address issues, implement updates, and ensure ongoing functionality. Evaluation criteria include the responsiveness to user feedback, timely bug fixes, and efficient handling of change requests.

2. **Explain Coupling in modular design with its types.**

Coupling is a measure that defines the level of inter-dependability among modules of a program. It tells at what level the modules interfere and interact with each other. The lower the coupling, the better the program. There are five levels of coupling, namely –

- **Content coupling** - When a module can directly access or modify or refer to the content of another module, it is called content level coupling.
- **Common coupling-** When multiple modules have read and write access to some global data, it is called common or global coupling.
- **Control coupling-** Two modules are called control-coupled if one of them decides the function of the other module or changes its flow of execution.
- **Stamp coupling-** When multiple modules share common data structure and work on different part of it, it is called stamp coupling.

- **Data coupling-** Data coupling is when two modules interact with each other by means of passing data (as parameter). If a module passes data structure as parameter, then the receiving module should use all its components.

2. **Explain Cohesion in modular design with its types.**

Cohesion Cohesion is a measure that defines the degree of intra-dependability within elements of a module. The greater the cohesion, the better is the program design. There are seven types of cohesion, namely –

- **Co-incidental cohesion** - It is unplanned and random cohesion, which might be the result of breaking the program into smaller modules for the sake of modularization. Because it is unplanned, it may serve confusion to the programmers and is generally not-accepted.
- **Logical cohesion -** When logically categorized elements are put together into a module, it is called logical cohesion.
- **Temporal Cohesion -** When elements of module are organized such that they are processed at a similar point in time, it is called temporal cohesion.
- **Procedural cohesion -** When elements of module are grouped together, which are executed sequentially in order to perform a task, it is called procedural cohesion.
- **Communicational cohesion** - When elements of module are grouped together, which are executed sequentially and work on same data (information), it is called communicational cohesion.
- **Sequential cohesion** - When elements of module are grouped because the output of one element serves as input to another and so on, it is called sequential cohesion.
- **Functional cohesion -** It is considered to be the highest degree of cohesion, and it is highly expected. Elements of module in functional cohesion are grouped because they all contribute to a single well-defined function. It can also be reused.

2. **Classified the software project planning and illustrate the software project scheduling.**

Software Project Planning and Scheduling

Software project planning involves defining the project scope, objectives, tasks, resources, and timelines. It sets the groundwork for project success by establishing a clear roadmap. Project scheduling, on the other hand, is the process of allocating resources and assigning timeframes to project tasks.

-Classification of Software Project Planning:

-Scope Definition: Clearly defining the project's scope is crucial to avoid scope creep and ensure all requirements are captured.

-Objective Setting: Establishing project objectives helps in aligning the team towards common goals.

-Task Identification: Identifying and listing all tasks required for project completion.

-Resource Planning: Allocating human and material resources based on task requirements.

-Risk Management: Identifying potential risks and developing strategies to mitigate them.

-Budgeting: Estimating and allocating financial resources required for the project.

-Illustration of Software Project Scheduling:

-Task Sequencing: Determining the order in which tasks should be performed.

-Resource Assignment: Allocating team members and tools to specific tasks.

-Timeline Establishment: Setting realistic timelines for each task, considering dependencies.

-Milestone Definition: Establishing key milestones to track progress.

-Progress Monitoring: Regularly tracking and updating the project schedule to ensure alignment with the overall project plan.

3. **Explain Data architectural and procedural Design in Software Design**

Data Architectural and Procedural Design in Software Design:

- Data Architectural Design: This focuses on defining the structure of the data within the system. It involves decisions on data organization, storage, retrieval, and relationships. Evaluation criteria include the efficiency of data access, data integrity, and adherence to database design principles.

- Procedural Design: Procedural design involves defining the procedures or algorithms that transform the input data into the desired output. Evaluation includes assessing the clarity, efficiency, and maintainability of the procedures.

4. **Explain the object-oriented Design concept in object-oriented Analysis.**

Object-Oriented Design Concept in Object-Oriented Analysis:

In object-oriented design, the concepts identified during object-oriented analysis are transformed into a detailed design. This includes the definition of classes, methods, and the relationships between objects. Evaluation criteria involve the clarity of class hierarchies, the effectiveness of encapsulation, and the adherence to object-oriented principles like inheritance and polymorphism.

5. **Explain major elements of DED & CFD,**

Major Elements of DED & CFD:

- DED (Data Entity Diagram): DED represents the data entities within a system and their relationships. Major elements include entities (representing real-world objects), attributes (characteristics of entities), and relationships (connections between entities).

- CFD (Control Flow Diagram): CFD illustrates the flow of control within a system, depicting the sequence of operations. Major elements include processes (tasks or

functions), control flow arrows (indicating the flow of control), and external entities (sources or destinations of data).

6. Use Case Diagram and State Diagram in the Context of UML:
   - Use Case Diagram: Use case diagrams illustrate the interactions between users and a system. Elements include actors (users or external systems), use cases (representing system functionalities), and associations (showing relationships between actors and use cases).

   - State Diagram: State diagrams represent the various states a system or object can exist in and transitions between these states. Elements include states, transitions, and events triggering state changes.

7. **Explain the use case diagram and state diagram in the context of UML.**

UML Diagrams in Online Examination System:

- Activity Diagram: This illustrates the flow of activities within the online examination system, such as user registration, question retrieval, and result submission.
- Class Diagram: Represents the classes in the system, like "Student," "Exam," and their relationships, attributes, and methods.

![Class Diagram](https://example.com/class_diagram.png)

In conclusion, the effective execution of software life cycle stages, robust project planning and scheduling, thoughtful data and procedural design, and the application of object-oriented design concepts are essential for successful software development. Utilizing UML diagrams like DED, CFD, Use Case, and State Diagrams further enhances the understanding and communication of complex systems. The example of an Online Examination System provides a practical context for applying these concepts in real-world scenarios.

8. Object-Oriented Design Concept in Object-Oriented Analysis:

In the realm of software development, the transition from object-oriented analysis (OOA) to object-oriented design (OOD) is a critical step where the conceptual model is transformed into a detailed design ready for implementation. OOD builds upon the principles identified during OOA, incorporating design patterns and strategies to create a robust and maintainable software solution.

- Modularity: One of the fundamental concepts in OOD is modularity. This involves breaking down the system into smaller, self-contained modules or classes. Each class encapsulates a specific set of functionalities, promoting reusability and maintainability. The modular approach allows developers to focus on individual components, making the design more comprehensible.

- Encapsulation: Encapsulation is a key principle in OOD that involves bundling data (attributes) and methods (behaviors) into a single unit known as a class. This encapsulation ensures that the internal workings of a class are hidden from external entities, fostering data

security and code organization. It also facilitates the creation of a well-defined interface for interacting with the class.

- Inheritance: Inheritance is a mechanism that allows a new class (subclass or derived class) to inherit attributes and behaviors from an existing class (superclass or base class). This promotes code reuse and establishes an "is-a" relationship between classes. Inheritance contributes to the creation of a class hierarchy, where common functionalities are defined in the base class and specialized features are added in derived classes.

- Polymorphism: Polymorphism is a concept that allows objects of different classes to be treated as objects of a common base class. This flexibility enables the same method to be used for different types of objects, simplifying code and enhancing adaptability. Polymorphism is achieved through method overriding and method overloading, allowing a single interface to serve multiple implementations.

9. **UML Diagrams in Online Examination System:**

a) Activity Diagram:

An activity diagram is a visual representation of the flow of activities within a system. In the context of an Online Examination System, the activity diagram captures the sequential steps involved in various processes.

- User Registration:
- The user initiates the registration process.
- System prompts for necessary details.
- Validation of information.
- Successful registration or correction if needed.

- Taking an Exam:
- User selects and initiates the exam.
- System presents questions.
- User responds to each question.
- Process repeats until all questions are answered.

- Result Submission:
- User completes the exam.
- System calculates the score. - User submits the result.

b) Class Diagram:

A class diagram provides a structural view of the system, illustrating the classes, their attributes, methods, and relationships.

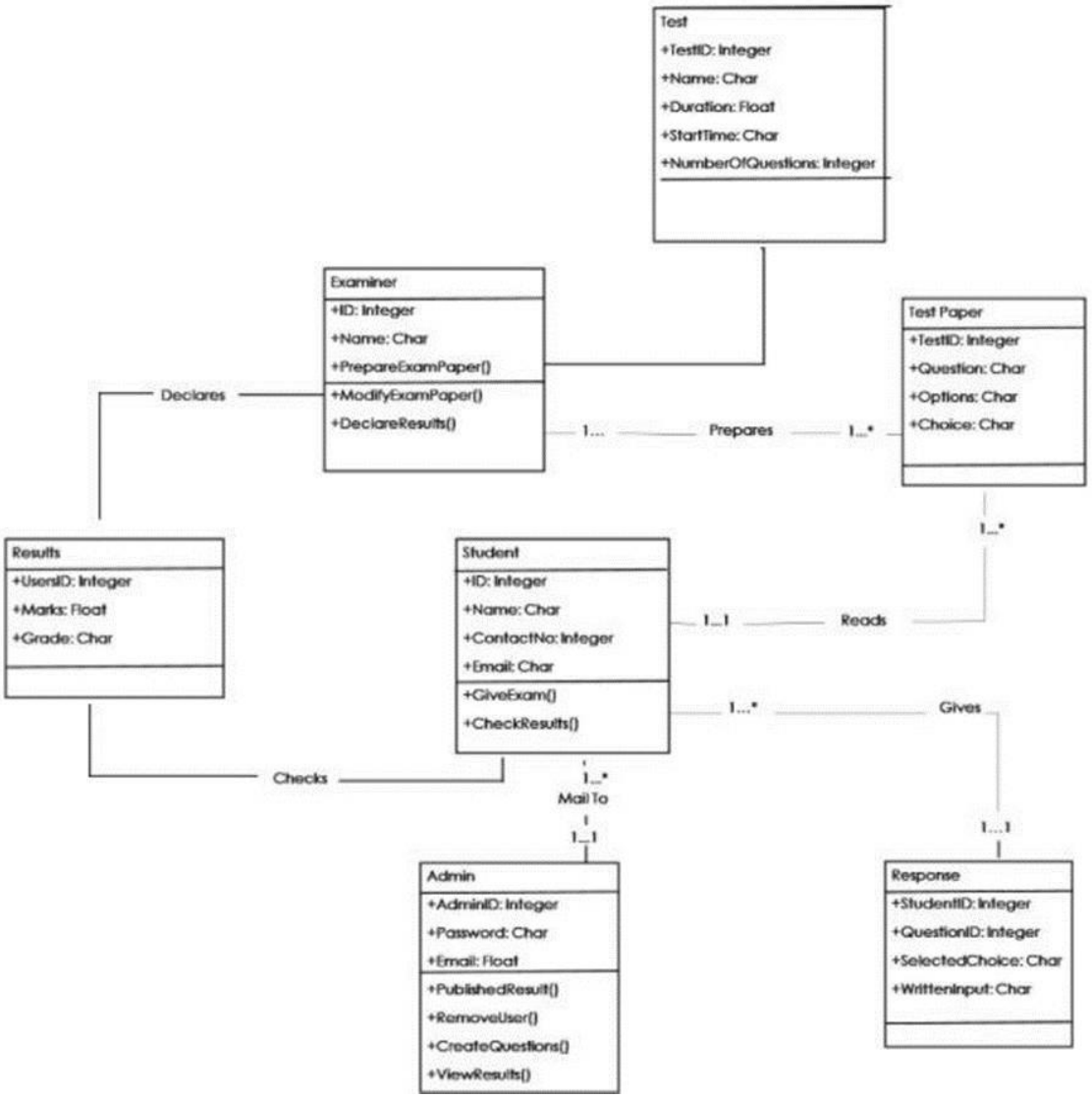
- Student Class:
- Attributes: username, password, personal information. - Methods: register(), logIn(), takeExam().

- Exam Class:
- Attributes: examID, questions, timeLimit.
- Methods: generateQuestions(), startExam().

- Result Class:
- Attributes: score, examID, studentID.
- Methods: submitResult(), calculateScore().

Relationships:
- Association between Student and Exam: A student can take multiple exams.
- Association between Student and Result: A student can have multiple results.

These UML diagrams provide a visual representation of the activities and class structures within the Online Examination System, aiding in communication and understanding during the software development process.


Test
+TestID: Integer
+Name: Char
+Duration: Float
+StartTime: Char
+NumberOfQuestions: Integer
Examiner
+ID: Integer
+Name: Char
+PrepareExamPaper()
+ModifyExamPaper()
+DeclareResults()
Declares
1...
Prepares
1...*
Test Paper
+TestID: Integer
+Question: Char
+Options: Char
+Choice: Char
1...*
Results
+UsersID: Integer
+Marks: Float
+Grade: Char
Student
+ID: Integer
+Name: Char
+ContactNo: Integer
+Email: Char
+GiveExam()
+CheckResults()
1...1
Reads
1...*
Gives
Checks
1...*
Mail To
1...1
Admin
+AdminID: Integer
+Password: Char
+Email: Float
+PublishedResult()
+RemoveUser()
+CreateQuestions()
+ViewResults()
1...1
Response
+StudentID: Integer
+QuestionID: Integer
+SelectedChoice: Char
+WrittenInput: Char